

Name: D. Gnana Deepthi

Reg No: 192411123

Marks: /100

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total
Marks: 50			
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results.	BL3 – Apply	5

2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly.	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view.	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation.	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints.	BL3 – Apply	5
6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance.	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error.	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table.	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes.	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage.	BL3 – Apply	5

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D. Gnanadeepthi.

ANSWERS

1.

The error is usually caused by an **incorrect JOIN condition** (or missing JOIN condition), which creates duplicate rows because each row from one table matches multiple rows from the other table.

Incorrect Query

```
SELECT s.student_id, s.student_name, c.course_name
FROM Students s
JOIN Enrollments e
JOIN Courses c
ON e.course_id = c.course_id;
```

Error:

- Students and Enrollments are not joined using a matching condition.
- This results in a Cartesian product (many duplicate rows).

Correct Query

```
SELECT s.student_id, s.student_name, c.course_name
FROM Students s
JOIN Enrollments e
ON s.student_id = e.student_id
JOIN Courses c
```

ON e.course_id = c.course_id;

Explanation

- Students is joined with Enrollments using student_id.
- Enrollments is joined with Courses using course_id.
- Proper JOIN conditions eliminate duplicate results and return only the matching records.

2.

A **correlated subquery** must reference a column from the outer query. If that correlation is missing, the subquery is evaluated independently and returns incorrect results.

Incorrect Query

```
SELECT emp_id, emp_name, salary
FROM Employee e
WHERE salary > (
    SELECT AVG(salary)
    FROM Employee
);
```

Error:

- The subquery calculates the average salary of **all employees** instead of the average salary within each department.
- It is not correlated with the outer query.

Correct Query (Correlated Subquery)

```
SELECT emp_id, emp_name, salary
FROM Employee e
WHERE salary > (
    SELECT AVG(salary)
    FROM Employee
    WHERE department = e.department
);
```

Explanation

- e.department refers to the department of the current employee in the outer query.
- The subquery calculates the **average salary for that employee's department**.
- The query correctly returns employees whose salary is greater than the average salary of their own department.

3.

Incorrect View

```
CREATE VIEW Student_Details AS
SELECT student_id, student_name, department
FROM Student
WHERE total_marks > (
    SELECT AVG(total_marks)
    FROM Student
    GROUP BY department
);
```

Issue

- The subquery returns **multiple rows** because of GROUP BY department.
- The > operator expects a **single value**, causing an error when the view is queried.

Correct and Optimized View

```
CREATE VIEW Student_Details AS
SELECT student_id, student_name, department, total_marks
FROM Student
WHERE total_marks > (
    SELECT AVG(total_marks)
    FROM Student
);
```

Explanation

- The subquery now returns a **single average value**.
- The view executes without errors and returns students whose marks are above the overall average.

- This version is simpler, efficient, and avoids the multi-row subquery error.

4.

Incorrect Relational Algebra Expression

Student $\bowtie$ Course

Error

- A **natural join** works only when both relations have a **common attribute with the same name**.
- If Student and Course do not share a common attribute (or the common attribute names differ), the natural join produces incorrect results or behaves like a Cartesian product.

Correct Relational Algebra Expression

Assume the common attribute is course_id:

Student $\bowtie$ Student.course_id = Course.course_id Course

or, if both relations already have the same attribute name (course_id):

Student $\bowtie$ Course

Explanation

- Ensure both relations have a matching join attribute.
- If attribute names differ, first rename them or use a **theta join** with an explicit condition.
- The corrected expression joins only matching tuples, eliminating incorrect or duplicate results.

5.

Incorrect Query

```
INSERT INTO Orders (order_id, customer_id, amount)
```

```
VALUES (101, 5001, 2500);
```

Violation

- customer_id = 5001 does **not exist** in the Customer table.
- This violates the **FOREIGN KEY (referential integrity)** constraint.

Correct Statements

-- Parent table

```
INSERT INTO Customer (customer_id, customer_name)
VALUES (5001, 'Rahul');
```

-- Child table

```
INSERT INTO Orders (order_id, customer_id, amount)
VALUES (101, 5001, 2500);
```

Foreign Key Constraint

```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    amount DECIMAL(10,2),
    FOREIGN KEY (customer_id)
    REFERENCES Customer(customer_id)
);
```

Explanation

- The parent record (Customer) must exist before inserting the child record (Orders).
- The FOREIGN KEY constraint ensures that every customer_id in Orders matches an existing customer_id in Customer, thereby maintaining referential integrity.

6.

Inefficient Query (Using Nested Subquery)

```
SELECT student_name
FROM Student
WHERE dept_id IN (
    SELECT dept_id
    FROM Department
    WHERE dept_name = 'CSE'
```

);

Optimized Query (Using JOIN)

```
SELECT s.student_name  
FROM Student s  
JOIN Department d  
ON s.dept_id = d.dept_id  
WHERE d.dept_name = 'CSE';
```

Explanation

- The original query uses a nested subquery, which can be less efficient on large tables.
- The JOIN directly matches rows from Student and Department using dept_id.
- Using JOIN improves readability and generally provides better performance, especially when appropriate indexes exist on the join columns.

7.

Incorrect Query

```
SELECT department, employee_name, AVG(salary)  
FROM Employee  
GROUP BY department;
```

Error

- employee_name is selected but **not included in the GROUP BY clause** and is **not an aggregate function**.
- This causes an error (or unpredictable results in some SQL modes) because there may be multiple employee names in each department.

Correct Query

```
SELECT department, AVG(salary) AS avg_salary  
FROM Employee  
GROUP BY department;
```

Explanation

- GROUP BY department groups all employees by department.
- AVG(salary) calculates the average salary for each department.

- Only the grouped column (department) and aggregated values (AVG(salary)) should appear in the SELECT list unless all non-aggregated columns are also included in the GROUP BY clause.

8.

Incorrect Query

UPDATE Employee

SET salary = salary + 5000;

Error

- The query **does not contain a WHERE clause**.
- As a result, **every row** in the Employee table is updated instead of only the intended records.

Correct Query

UPDATE Employee

SET salary = salary + 5000

WHERE department = 'IT';

Explanation

- The WHERE clause restricts the update to employees in the **IT** department.
- Without a WHERE clause, SQL updates **all rows** in the table, which can lead to unintended data changes.

9.

Inefficient Query (Using Subqueries)

```
SELECT employee_name
FROM Employee
WHERE dept_id IN (
    SELECT dept_id
    FROM Department
    WHERE location = 'Chennai'
)
```

```
AND emp_id IN (  
    SELECT emp_id  
    FROM Project  
    WHERE project_name = 'ERP'  
);
```

Optimized Query (Using JOINS)

```
SELECT e.employee_name  
FROM Employee e  
JOIN Department d  
ON e.dept_id = d.dept_id  
JOIN Project p  
ON e.emp_id = p.emp_id  
WHERE d.location = 'Chennai'  
AND p.project_name = 'ERP';
```

Recommended Indexes

```
CREATE INDEX idx_employee_dept  
ON Employee(dept_id);
```

```
CREATE INDEX idx_project_emp  
ON Project(emp_id);
```

```
CREATE INDEX idx_department_location  
ON Department(location);
```

Explanation

- Replaced nested subqueries with **JOINS** to reduce query complexity.
- JOINS allow the database optimizer to generate a more efficient execution plan.
- Indexes on dept_id, emp_id, and location speed up joins and filtering, improving performance on large tables.

10.

Incorrect Tuple Relational Calculus Expression

$\{ s \mid \text{Student}(s) \wedge \forall c (\text{Course}(c) \rightarrow s.\text{course_id} = c.\text{course_id}) \}$

Error

- The **universal quantifier ($\forall$)** requires a student's `course_id` to match **every course** in the `Course` relation.
- Since a student cannot usually be enrolled in every course, the condition is false for all tuples, resulting in an **empty result set**.

Correct Expression

$\{ s \mid \text{Student}(s) \wedge \exists c (\text{Course}(c) \wedge s.\text{course_id} = c.\text{course_id}) \}$

Explanation

- Replace the **universal quantifier ($\forall$)** with the **existential quantifier ($\exists$)**.
- $\exists c$ checks that there **exists at least one course** whose `course_id` matches the student's `course_id`.
- The corrected expression returns all students who are enrolled in a valid course instead of an empty result set.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

		documentatio n			
--	--	-------------------	--	--	--